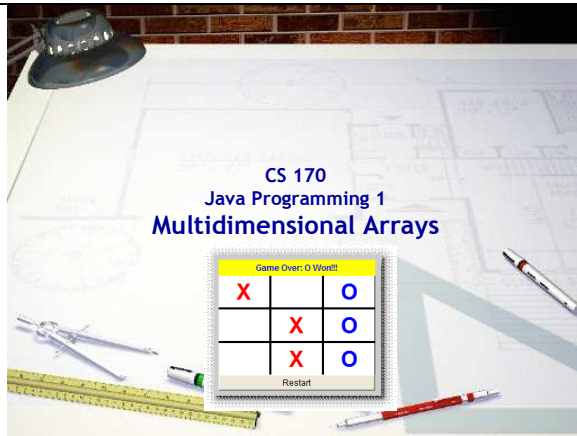


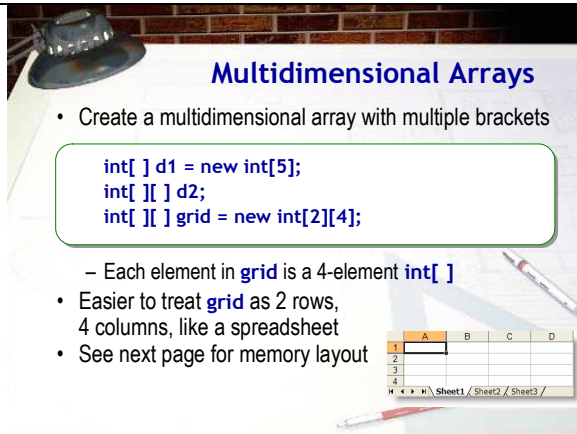


Slide 1
CS 170
Java Programming 1
Multidimensional Arrays
 Duration: 00:00:39
 Advance mode: Auto

Notes:

Hi Folks. Welcome to the CS 170, Java Programming 1 lecture on Multidimensional arrays.

When you declare an array, the elements can be of any type. Java also allows you to create array variables that refer, not to Button objects or int values, but to other arrays. These are called multidimensional arrays. Since we'll only be working with two dimensions in this class, I'll refer to these as 2D arrays, although you can have as many dimensions as you like. You can think of them as arrays that contain rows and columns like the Tic-Tac-Toe board shown here.



Slide 2
Multidimensional Arrays
 Duration: 00:01:37
 Advance mode: Auto

Notes:

To create a 2D array, just extend the syntax you've already learned for 1D arrays. For instance, in the example shown here: `int[] d1 = new int[5];` the variable `d1` is an `int[]` (int array) that refers to 5 int values.

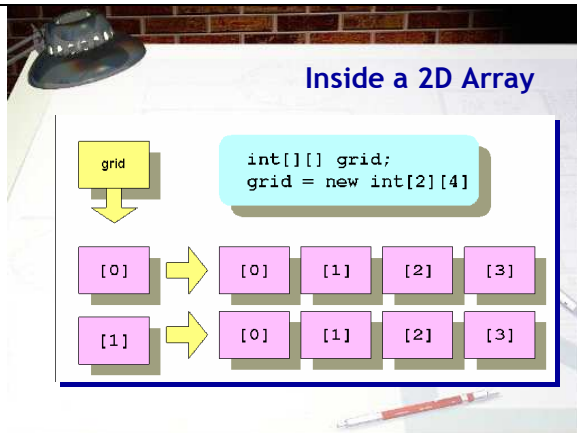
Since that's true, perhaps you can figure out what the code shown here means: `int[][] d2;`

The variable `d2` is an array variable, just the like the variable `d1` in the first example. But, each of the elements contained in the `d2` array is not an int, but an `int[]`— that is, an int array variable.

Of course, just like with 1D arrays, an array variable doesn't actually "point to" anything, until you allocate some memory by using `new` and an array constructor. The syntax for doing this with a 2D array is very similar to that used for a single-dimension array: `int[][] grid = new int[2][4];`

This piece of code creates a variable (`grid`), that refers to a two-element array. Each element in that two-element array (`grid[0]` and `grid[1]`) is an `int[]`, that is, an int array variable.

The easiest way to think of a 2D array is to think of rows and columns. A spreadsheet is a useful mental model because it's composed of rows and columns. In reality, though, a 2D array is composed of arrays of arrays. On the next slide you'll see what that really looks like.



Slide 3
Inside a 2D Array
Duration: 00:00:39
Advance mode: Auto

Notes:

Here's a memory layout that shows you what things look like in memory.

- The statement `int[][] grid` creates an array variable that doesn't refer to any array elements.
- The statement `grid = new int[2][4]` creates a two element array containing array variables, and then "points" the grid variable to those elements. Since each element in grid is an array variable, `grid[0]` points to one array and `grid[1]` points to another four-element array.

Using 2D Elements

- Don't really have to think about **physical** storage
 - Just treat elements as a big grid
- Address each element using two subscripts

```
grid[ 1 ][ 3 ] = 30;
```

- First subscript is the row, second is the column
- Puts value 30 in last element of second row
- Remember: arrays are numbered 0 to length-1

Slide 4
Using 2D Elements
Duration: 00:01:30
Advance mode: Auto

Notes:

Rather than thinking of 2-D arrays as shown in the previous illustration, (even though that is how the arrays are actually implemented in memory), it is usually easier to think of them as an actual grid composed of rows and columns, like a spreadsheet. The only difference is that a spreadsheet has columns labeled A, B, C and so on, and rows starting at 1. In a 2-D array, both the columns and rows used numbers and the first column and the first row are numbered starting at 0.

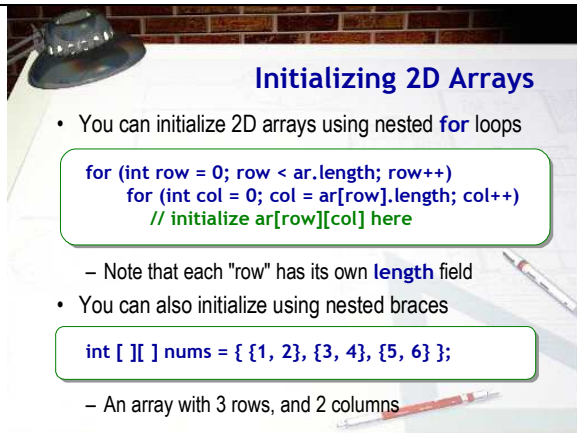
Thinking of 2-D arrays in this way makes it easy to remember where the individual elements are located.

To access an individual element, instead of providing one subscript enclosed in brackets, you supply two subscripts, each enclosed in its own set of brackets. (You cannot put both subscripts in the same set of brackets, as you can do in Pascal, or C#).

- The first subscript represents the row in the array.
- The second subscript represents the column of the element you want.

Remember that both subscripts are zero based.

If you know a little C or C++, I should warn you that in Java, subscripts do not "wrap around" as they do in those languages. You cannot access `scores[1][0]` by using the notation `scores[0][3]`. That's because, in reality, you are accessing an individual array variable, as shown in the earlier illustration.



Initializing 2D Arrays

- You can initialize 2D arrays using nested **for** loops

```
for (int row = 0; row < ar.length; row++)  
    for (int col = 0; col = ar[row].length; col++)  
        // initialize ar[row][col] here
```

- Note that each "row" has its own **length** field
- You can also initialize using nested braces

```
int [ ][ ] nums = { {1, 2}, {3, 4}, {5, 6} };
```

- An array with 3 rows, and 2 columns

Slide 5 

Initializing 2D Arrays

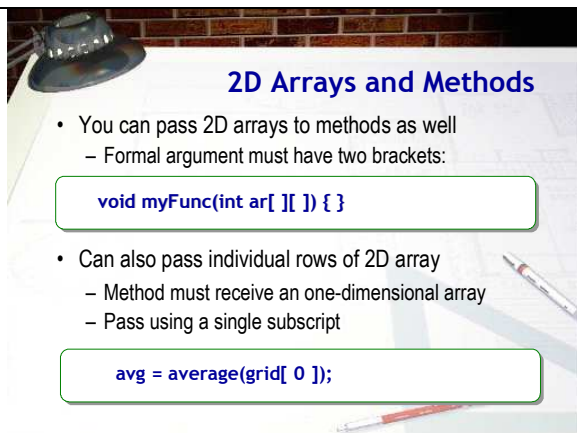
Duration: 00:01:16

Advance mode: Auto

Notes:

You can initialize 2D arrays by using loops and storing values in each element, just as you can with a one-dimensional array. Of course, when accessing 2D arrays, you'll usually use a pair of nested for loops with indexes for the row and column. Notice when you do this, each row has its own length field. Of course if you instantiate your array using the notation you've seen, all of the rows will have the same length. It's possible, though to instantiate each row separately, thus ending up with a different number of columns in each row. I won't ask you to do that.

Just as with 1D arrays, you can create and initialize a 2D array in one statement by providing a list of initial values in an initializer list. When you do this, you supply a separate initializer list for each of the subarrays. Each of these sub-array initializers is enclosed in its own set of braces, and separated from the others by commas. Here's an example that creates a 3 x 2 array of int variables. When you use automatic initialization like this, you must remember to match the number of brackets and initializers.



2D Arrays and Methods

- You can pass 2D arrays to methods as well
 - Formal argument must have two brackets:

```
void myFunc(int ar[ ][ ]) { }
```

- Can also pass individual rows of 2D array
 - Method must receive an one-dimensional array
 - Pass using a single subscript

```
avg = average(grid[ 0 ]);
```

Slide 6 

2D Arrays and Methods

Duration: 00:00:55

Advance mode: Auto

Notes:

The way that Java 2D arrays are implemented makes it very easy to write methods which take 2D arrays as arguments. You just have to remember to declare the formal arguments using two brackets instead of one.

Here's the header for a void method that takes a 2D array of ints.

Since each 2D array is actually a 1D array containing other 1D array elements, you can pass those elements, which represent the individual rows of the array, to a method that processes 1D arrays. To pass a single row, your actual parameter should use a single subscript like this. If you wanted to pass the entire array, you wouldn't use a subscript at all. To pass a single int value in the array, you'd use a pair of subscripts.

OK, that's the 2D theory. Let's try some exercises.



Express Yourself

- **Exercise 1:** Open the file **Array2DPractice** and consider the following array defined in **run()**:


```
- int[][] values = { {2, 5}, {6, 7}, {3, 9} };
```
- Write a method named **average()** that returns the average of all the numbers contained in the array.
- Uncomment the printing code in **run()**, run and shoot a screen-shot.

Slide 7

Express Yourself

Duration: 00:00:48

Advance mode: Auto

Notes:

Start a new section in your IC document. Then open the file **Array2DPractice** which contains a 2D int array named **values** declared like this in the **run()** method.

For Exercise 1, define a method named **average** that takes a single 2D int array parameter. The method should average all of the numbers contained in the array and return the average.

Once you've added the **average()** method to the **Array2DPractice** class, uncomment the code that calls **average** and prints out the returned value. Compile and run the program, and shoot me a screenshot of the results.



Express Yourself

- **Exercise 2:** Consider the following array:


```
- int[][] values = { {2, 5}, {6, 7}, {3, 9} };
```
- Write a second method named **average()** that returns the average of all the numbers in only **one row** of the array
- Then, add a statement to **run()** passing the second row to your method and printing the result.

Slide 8

Express Yourself

Duration: 00:00:43

Advance mode: Auto

Notes:

OK, now you've written an **average** method that can average the numbers in a 2D int array. Now, add a second **average** method to the class that only averages the elements of a 1D int array. In the **run** method, add a statement that passes the second row from the **values** array to the new **average** method and then prints the result. (The easiest way to do that is to copy and modify the code used to print the output for Exercise 1.)

Run it and snap a screenshot. Paste the picture into your IC document as Exercise 2.



Express Yourself

- **Exercise 3:** Consider the following array:


```
- int[][] values = { {2, 5}, {6, 7}, {3, 9} };
```
- Write a third method named **average()** that returns the average of all the numbers in **one column** of the array. Pass the column as the second argument.
- Write a statement that calls your method to print the average of the second column in **values**.

Slide 9

Express Yourself

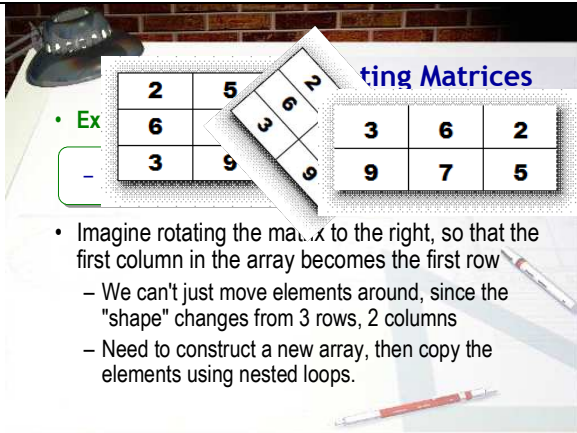
Duration: 00:00:43

Advance mode: Auto

Notes:

Let's try another **average** method, one that averages all of the numbers in one column of the array. For this version of **average**, you'll pass two parameters: the 2D array variable and the column number to average. Then, in the body of your method, just sum and then average the numbers in the specified column.

Once you've written this third **average** method, add a statement to **run()** that calls the method and prints the average of the values in the second column of **values**. Run it and shoot a screenshot. Paste it into your IC document as Exercise 3.



• Ex

- Imagine rotating the matrix to the right, so that the first column in the array becomes the first row
 - We can't just move elements around, since the "shape" changes from 3 rows, 2 columns
 - Need to construct a new array, then copy the elements using nested loops.

Slide 10

Rotating Matrices

Duration: 00:01:54
Advance mode: Auto

Notes:

Now let's try something a little more difficult. Here's the same array, values, that we saw before.

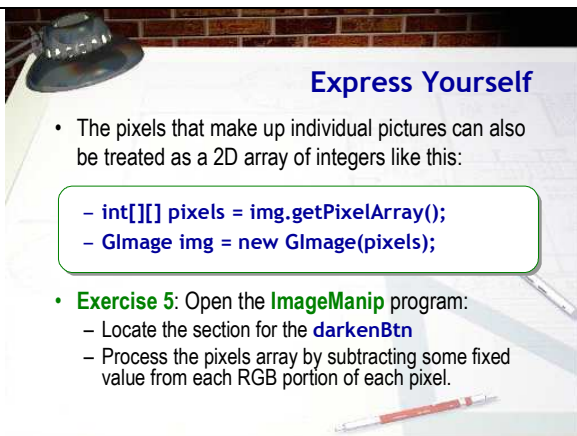
If you picture this in your mind, you can see the first row containing 2 and 5. Underneath the two and five are the second row 6 and 7, and underneath that, in the third row are 3 and 9 like this.

Now imagine we push this "block" over on its side like this so that the first column becomes the first row of the new array. Notice that we can't just move elements around, since the shape of the finished array changes from 3 x 2 to 2 x 3.

To do this, write a method named rotateRight. As you can see from the Javadoc I've added for the method in Eclipse, it takes a single 2D array as a parameter, and returns the new array. Here's what you need to put inside it.

- Create a temporary array in the method. Use the length of the first array as the number of columns for the temp array. Use the length of the first row in the original array (that is, the columns from the original array) as the number of rows in the temp array.
- Now, write a nested loop that processes the first array and copies the values into the second array. Notice that if process the first array column by column, starting at the bottom (the 3), you can copy the values across to the first row.

When you're done, add one final print statement to the run method that rotates values and prints out the values in the new array. Make sure that values is unchanged after you do this. Run and paste into your IC document as Exercise 4.



Express Yourself

- The pixels that make up individual pictures can also be treated as a 2D array of integers like this:


```
- int[][] pixels = img.getPixelArray();
- GImage img = new GImage(pixels);
```
- Exercise 5:** Open the ImageManip program:
 - Locate the section for the darkenBtn
 - Process the pixels array by subtracting some fixed value from each RGB portion of each pixel.

Slide 11

Express Yourself

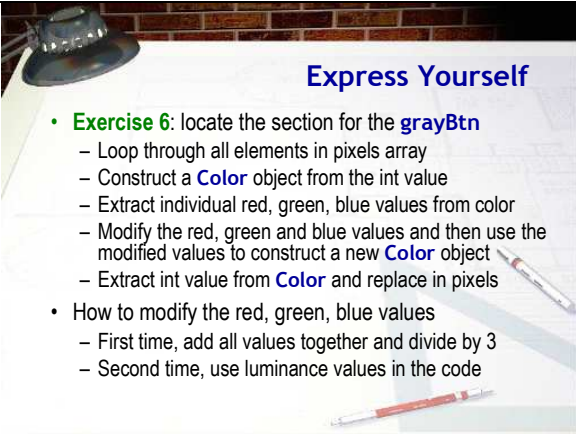
Duration: 00:01:59
Advance mode: Auto

Notes:

If you look inside a photographic image, what you'll find is a 2D array of integers. With the GImage class in the ACM libraries, we can extract those pixels using the getPixelArray() method like this. Then, we can use a loop to modify the pixels we're interested in. Once we've modified the array, we can turn it back into a new image using one of the GImage constructors, like this.

For the last few exercises, let's do something fun, so you can get a feel for how 2D arrays are actually used in the real world. To do that, we'll write a few little image filter programs, like the filters found in programs like PhotoShop. Open the file ImageManip.java, and then locate the section that handles the code for the darken button. (I've added a #1 comment). The code in this section will be called when the user clicks the program's Darken button. Here's how to make it work.

I've already handled initializing the pixel array and have taken care of reconstructing and displaying the image after you modify the array. All you have to do is write a loop that visits

	every element in the array pixels. Inside the body of the loop: • construct a local Color object, passing the pixel element to the one-int-parameter constructor. • call the Color object's darker() method and save the result back in the Color object • call the Color object's getRGB() method to extract the int value from the Color object and store it back into the pixels array. When you've made these changes, run the program and click the Darken button a few times to make sure it works. Then, paste your code into your IC document for Exercise 5.
 Slide 12 🎧 Express Yourself Duration: 00:01:21 Advance mode: Auto • Exercise 6: locate the section for the grayBtn – Loop through all elements in pixels array – Construct a Color object from the int value – Extract individual red, green, blue values from color – Modify the red, green and blue values and then use the modified values to construct a new Color object – Extract int value from Color and replace in pixels • How to modify the red, green, blue values – First time, add all values together and divide by 3 – Second time, use luminance values in the code	Notes: For Exercise 6, follow the same pattern. Write a nested loop that visits every element in the pixels array pixels. Inside the body of the loop: • extract the int from the pixels array and use it to construct a Color object • use the getRed(), getGreen() and getBlue() methods to extract the red, green and blue portions from the Color object and then use those (after some modification) to construct a new Color object. I'll talk about how to modify those in a moment. • use the getRGB() method to place a new value in the pixels array. I want you to try two different methods for graying out the image. For the first, simply average the red, green and blue portions when constructing the new object. For the second, multiply each of the color values by the luminance factors I've supplied as comments, and then add the results together. Run each version and make sure it works. Then shoot me a screenshot and copy the code into your IC document for Exercise 6. (You'll have two screen-shots and two code fragments for Exercise 6.)

Express Yourself

- **Exercise 7:** locate the section for the **mirrorBtn**
 - Write a loop that visits half the columns
 - `int nCols = pixels[0].length / 2;`
 - Copy the elements from the left to the right side
 - `pixels[r][nCols - 1 - c] = pixels[r][c];`
- **Exercise 8:** locate the section for the **rotateBtn**
 - Use your code to rotate a matrix to the right
- **Exercise 9:** locate the section for the **shrinkBtn**
 - Write code to copy every other pixel to the output

Slide 13
Express Yourself
Duration: 00:02:33
Advance mode: Auto

Notes:

OK, now instead of modifying the pixel elements, let's try moving them around. For Exercise 7, we'll create a mirror image of the picture. Here's how. Locate the section that says **mirrorBtn** and then add code that:

- uses a nested loop to visit each element, but instead of visiting every column, go only halfway across. That is, if the length of the first row is 1000 pixels, go only to element 499. To get this number, which you should store in a variable like `nCols`, you can just divide `pixels[0].length` by 2.
- read each element at `pixels[r][c]` and copy the pixel to `pixels[r][nCols - 1 - c]`. This copies the first pixel in the row to the last pixel in the row, the second pixel to the second-to-last pixels and so on, creating a mirror effect.

Run the program and shoot me a screenshot. Then paste your code in for Exercise 7.

The last two exercises, 8 and 9, are "challenge" exercises, which may take you a little more time to finish. Instead of giving you specific instructions, I'll let you work them out. Remember, though, if you get stuck, you can always check on the solutions page.

For Exercise 8, I want you to rotate the pixels array to the right. You'll have to use the same kind of code you used for Exercise 4. Remember that to rotate an array, you'll first have to allocate a new array using the original array (pixel's) column value for the number of rows and the original array's row value for the number of columns. Then, you'll copy the values from the original array (pixels) to the appropriate position in the new array. If you've done Exercise 4 correctly, you'll use exactly the same code here. Finally, you need to assign the new array to the variable `pixels`.

Exercise 9 is similar, but this time I want you to shrink the picture. To do that allocate an array that has half the number of rows and columns as pixel. Then, loop through the pixels array visiting every other row and column and assign the pixel you find there to the new array.

When you're done, shoot me screenshots for both exercises and show me your code.